

Exercise 1: Setting Up JUnit

Scenario

This exercise demonstrates how to set up JUnit in a Java project to enable unit testing. JUnit is a widely used testing framework that helps developers verify the correctness of their code.

Objective

The objective of this exercise is to configure JUnit in a Maven-based Java project, add the required dependency, create a test class, and execute a simple unit test successfully.

1. Understanding JUnit Setup

JUnit is a testing framework for Java applications. It allows developers to write and execute automated test cases, ensuring that application code behaves as expected.

The setup process includes:

- Creating a Maven project
- Adding the JUnit dependency
- Creating a test class
- Running the test successfully

2. Implementation

Step 1: Add JUnit Dependency in pom.xml

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Step 2: Create SampleTest.java

```
import org.junit.Test;

public class SampleTest {

    @Test
    public void testHello() {
```

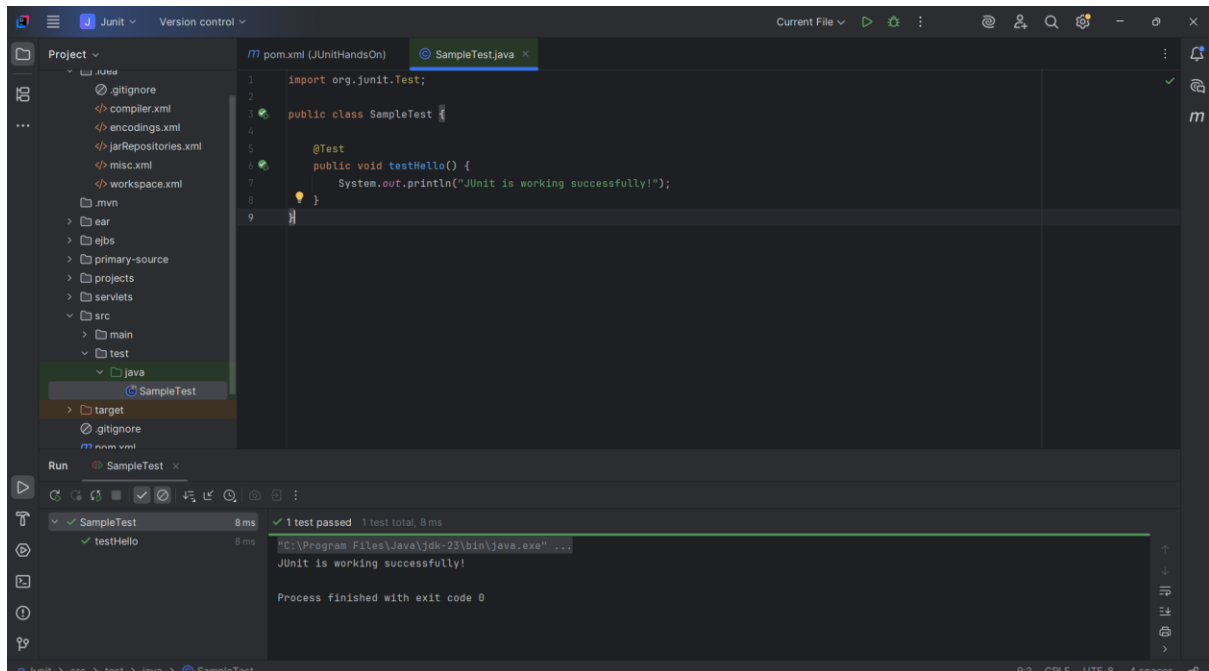
```
        System.out.println("JUnit is working successfully!");
    }
}
```

3. Output Screenshot

Expected Output:

- JUnit dependency loaded successfully.
- Test executed successfully.
- Green check mark displayed.
- Console Output:

JUnit is working successfully!



4. Conclusion

Successfully configured JUnit in a Maven project and executed the first unit test. This setup provides the foundation for writing automated unit tests in Java applications.

Exercise 3: Assertions in JUnit

Scenario

This exercise demonstrates the use of various JUnit assertions to validate expected results during unit testing.

Objective

The objective of this exercise is to understand different assertion methods provided by JUnit and verify program behavior using appropriate assertions.

1. Understanding Assertions

Assertions compare the expected result with the actual result produced by the program.

Common assertions include:

- `assertEquals()`
- `assertTrue()`
- `assertFalse()`
- `assertNull()`
- `assertNotNull()`

2. Implementation

AssertionsTest.java

```
import org.junit.Test;
```

```
import static org.junit.Assert.*;
```

```
public class AssertionsTest {
```

```
    @Test
```

```
    public void testAssertions() {
```

```
        assertEquals(5, 2 + 3);
```

```
        assertTrue(5 > 3);
```

```
        assertFalse(5 < 3);
```

```
        assertNull(null);
```

```
        assertNotNull(new Object());
```

```

        System.out.println("All assertions passed successfully!");
    }
}

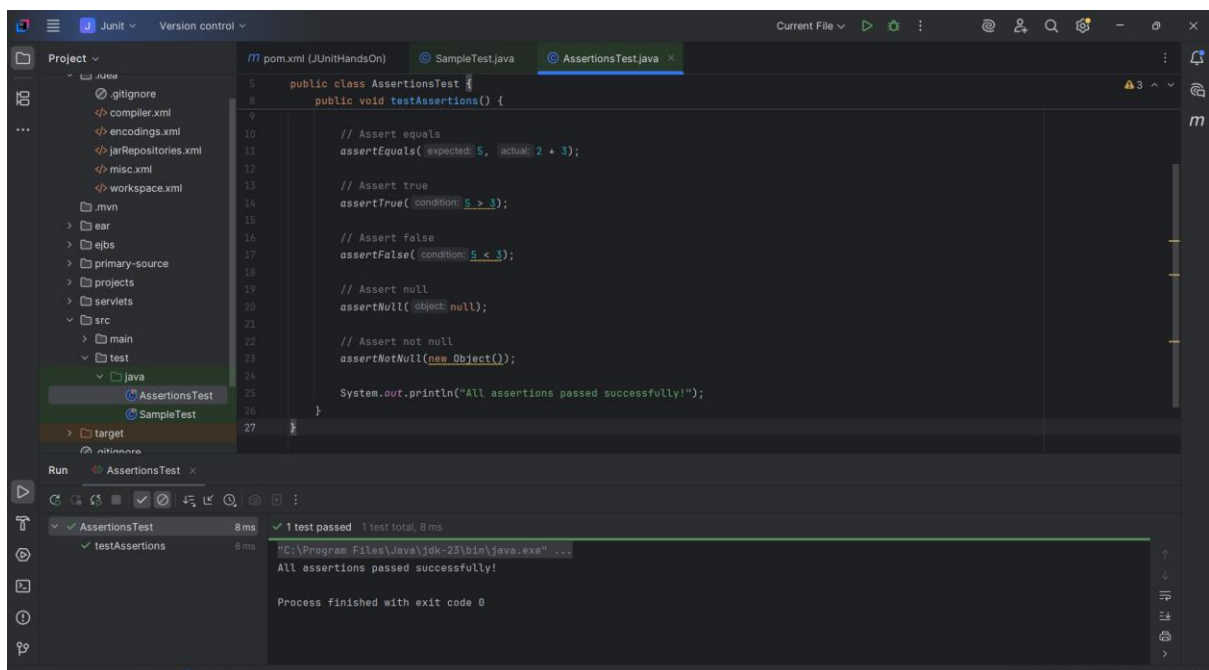
```

3. Output Screenshot

Expected Output:

- All assertions executed successfully.
- Green check mark displayed.
- Console Output:

All assertions passed successfully!



4. Conclusion

Successfully implemented various JUnit assertions to validate expected outputs. Assertions improve software reliability by automatically verifying program correctness during testing.

Exercise 4: Arrange-Act-Assert (AAA) Pattern, Test Fixtures, Setup and Teardown Methods in JUnit

Scenario

This exercise demonstrates the Arrange-Act-Assert (AAA) testing pattern along with the use of @Before and @After annotations to organize unit tests effectively.

Objective

The objective of this exercise is to write well-structured unit tests using the AAA pattern and understand how setup and teardown methods simplify test preparation and cleanup.

1. Understanding the AAA Pattern

The AAA pattern divides a test into three logical sections:

- Arrange – Prepare test data and objects.
- Act – Execute the method under test.
- Assert – Verify the expected result.

JUnit also provides:

- @Before – Executes before each test.
- @After – Executes after each test.

2. Implementation

Calculator.java

```
public class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

CalculatorTest.java

```
import org.junit.After;  
import org.junit.Before;  
import org.junit.Test;  
  
import static org.junit.Assert.assertEquals;  
  
public class CalculatorTest {  
  
    private Calculator calculator;  
  
    @Before  
    public void setUp() {  
        System.out.println("Setting up...");  
        calculator = new Calculator();  
    }  
}
```

```
}
```

```
@Test
```

```
public void testAddition() {
```

```
    // Arrange
```

```
    int a = 10;
```

```
    int b = 20;
```

```
    // Act
```

```
    int result = calculator.add(a, b);
```

```
    // Assert
```

```
    assertEquals(30, result);
```

```
}
```

```
@After
```

```
public void tearDown() {
```

```
    System.out.println("Cleaning up...");
```

```
}
```

```
}
```

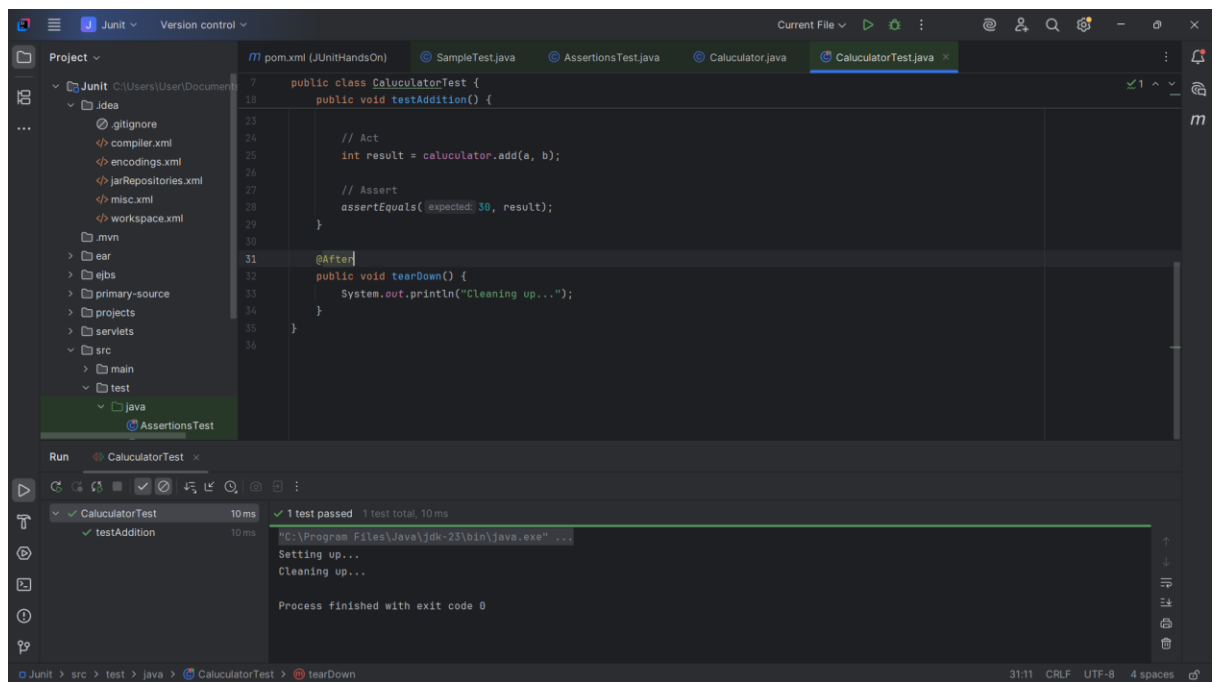
3. Output Screenshot

Expected Output:

Setting up...

Cleaning up...

- Test passed successfully.
- Green check mark displayed.



4. Conclusion

Successfully implemented the Arrange-Act-Assert testing pattern using JUnit. The use of `@Before` and `@After` improved test organization, while assertions verified the correctness of the program, resulting in clean, maintainable, and reliable unit tests.